DP-OPS-0022 Strategic Architecture Report: The Consolidation of the Project Factory and Resolution of Epistemic Drift

1. Executive Strategic Analysis

The Stela platform, operating at the specific freshness state of f2258f404 on the main branch, is currently undergoing a fundamental ontological metamorphosis. This transition, initiated by the partial implementation of Dispatch Packet DP-OPS-0021 ("The Autopoietic Project Binary"), aims to evolve the repository from a static file storage system into a dynamic "Project Factory"—a self-sustaining engine capable of manufacturing and verifying compliant software payloads through high-density governance. However, a rigorous forensic audit of the post-implementation state, cross-referenced against the "Stela Agent Constitution Refinement Discussion," reveals significant "Epistemic Drift"—a divergence between the intended governance model and the actualized codebase—that threatens the integrity of the system.

This comprehensive research report serves as the strategic foundation for **DP-OPS-0022**. Its primary objectives are threefold: to resolve the "Red Gate" anomaly currently compromising the Continuous Integration (CI) pipeline due to fragile path resolution logic; to correct the "Ontological Map" regarding the relationship between Agents, Tasks, and Skills; and to purge "Documentation Debt" that violates the Single Source of Truth (SSOT) doctrine.

Crucially, this analysis rejects the previous proposal to import agent definitions directly into the system root as a replacement for the generic "Contractor" persona. Such an approach constitutes a "Constitutional Violation," unmooring the AI agent from the non-negotiable behavioral constraints defined in AGENTS.md. Instead, this report establishes a strict "Constitution vs. Citizen" hierarchy, where AGENTS.md remains the supreme law, and specific agent files serve as job descriptions operating *within* those boundaries.

By executing the remediation strategies outlined herein, Stela will transition from a state of fragile experimentation to a robust "High-Governance" engine, enforced by "Green Gates Only" and a rigorous "Economy of Truth."

2. Theoretical Framework: The Epistemology of High-Governance

To rigorously define the requirements for DP-OPS-0022 and justify the rejection of the

previous agent injection strategy, it is necessary to deconstruct the epistemic mechanics that govern the Stela repository. The governance of an AI-driven repository is not merely a matter of file management; it is a problem of the "Political Economy of Truth." In an agentic environment, truth is a finite resource quantified in tokens, and the efficiency of the system is determined by the "Attention Voltage"—the capacity of the model to attend to relevant instructions amidst competing noise.

2.1 The Political Economy of Truth

The central axiom of the Stela architecture is the "Economy of Truth," which dictates that for a proposition to be valid, the resources required to derive it must be explicitly present in the context. This principle drove the "Great Compression" of TRUTH.md and AGENTS.md in Phase Two. However, the current state analysis reveals a dangerous conflation between "Economy" (optimization) and "Scarcity" (deprivation).

Epistemic Scarcity occurs when unresolvable unknowns prevent probabilistic optimization. In the context of Stela, this manifests when the removal of narrative justification strips away the semantic causal links required for the model to bridge the gap between "Instruction" and "Intent." The user's directive to treat AGENTS.md as a "Constitution" resolves this tension. A Constitution functions as a set of axioms; it does not need to argue for its own validity. By stripping the "Why" (narrative justification) from the "What" (operational rules), we increase the "Epistemic Density" of the context. This reduces the surface area for "Interpretation Drift," where an agent might hallucinate an exception to a rule based on a misunderstood rationale.

2.2 Autopoiesis and the Project Factory

The ultimate goal of Stela is Autopoiesis—the capability of the system to reproduce and maintain itself. A system is autopoietic if its maintenance processes are internal to its operation. The ops/bin/project utility is designed to be the primary engine of this autopoiesis, acting as a "Membrane" that selectively permeates context between the Platform (the OS) and the Payload (the Project).

This separation is vital because the code in projects/ has a different lifecycle, ownership, and truth table than the code in ops/. Indiscriminately loading "Project Agents" or "Project Tasks" into the "System Context" creates "Context Pollution." This risks a collapse into a "Vector Haze," where the model cannot distinguish between immutable system laws (e.g., "No contractions") and mutable project requirements (e.g., "Use Python 3.11"). The "Red Gate" anomaly discussed in Section 3 represents a failure of this autopoiesis: the system's internal verification tools failed to correctly identify their own location relative to the system root, leading to a breakdown in self-maintenance.

2.3 The Ontological Tension: System vs. Project

The current repository state exhibits "Ontological Tension"—confusion regarding the hierarchy of authority. The "Stela Agent Constitution Refinement Discussion" proposed a

model where specialized agent files (e.g., R-AGENT-06) would replace the generic "Contractor" definition. This analysis rejects that proposal as "Drift."

If the specific Agent file replaces the Constitution, the agent is no longer bound by the system's fundamental laws. A "Security Reviewer" might decide that using contractions is acceptable because their specific job description doesn't explicitly forbid it. This creates a fractured governance model where every agent creates its own micro-constitution. The corrected model, detailed in Section 4, enforces a "Constitution vs. Citizen" relationship: the Citizen (Agent) inherits the rights and restrictions of the Constitution (AGENTS.md) and cannot override them.

3. Forensic Audit of Repository State (f2258f404)

A granular, line-by-line forensic verification of the repository state (Freshness: f2258f404) against the requirements of the Stela standard reveals specific vulnerabilities and structural violations that mandate the creation of DP-OPS-0022.

3.1 The "Red Gate" Anomaly: Path Resolution Failure

The user reports that "repo-gates is red" due to "strange path reporting from scripts located in /tools." This represents a failure in "Anti-Drift Governance" and requires immediate remediation. A deep forensic analysis of the shell scripts in tools/ reveals a subtle but critical vulnerability in how path resolution is handled, particularly in Continuous Integration (CI) environments where the working directory and git state may differ from a local developer's environment.

The Diagnosis: Relative Path Fragility

The primary linter scripts—tools/lint/truth.sh, tools/lint/context.sh, and tools/lint/library.sh—rely on establishing the REPO_ROOT to locate canonical files like TRUTH.md or ops/lib/manifests/CONTEXT.md.

In the current dump (f2258f404), tools/lint/context.sh uses the following logic:

Bash

```
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE}")" && pwd)"
REPO_ROOT="$(cd "${SCRIPT_DIR}/.." && pwd)"
MANIFEST_PATH="${REPO_ROOT}/ops/lib/manifests/CONTEXT.md"
```

The Logic Failure:

1. The script tools/lint/context.sh is located in tools/lint/.
2. dirname "${BASH_SOURCE}" correctly identifies tools/lint (or a relative equivalent).
3. SCRIPT_DIR resolves to the absolute path /abs/path/to/tools/lint.
4. REPO_ROOT is defined as cd ${SCRIPT_DIR}/... This resolves to /abs/path/to/tools.
5. The script then constructs the path: /abs/path/to/tools/ops/lib/manifests/CONTEXT.md.

This path does not exist. The ops/ directory is located at the repository root, not inside tools/. The script assumes it is running from one level deep (e.g., tools/script.sh), but it is actually two levels deep (tools/lint/script.sh). This "Path Depth Mismatch" causes the script to fail to find the manifest, triggering the red gate.

Similarly, tools/verify.sh defines:

Bash

```
ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE}")/.." && pwd)"
```

Since verify.sh is in tools/ (Depth 1), going up one level (..) correctly takes it to the Repo Root. The linters in tools/lint/ (Depth 2) seemingly inherited this pattern but failed to account for the additional directory nesting. This fragility violates the "Anti-Drift Governance" directive in AGENTS.md.

The Fix: Git-Anchored Root Resolution

To permanently resolve this and prevent future drift if scripts are moved, the system must abandon relative path counting (../..) and rely on the definitive authority of git. In a "High-Governance" repository, Location Independence is a requirement. Scripts must verify their own execution context based on the "Truth" of the repository structure (git) rather than the "Reality" of the file system (pwd).

The proposed logic pattern ("The Anchor") utilizes git rev-parse --show-toplevel. This command, when executed within a git working tree, returns the absolute path to the root of the repository. This method is robust against depth changes; a script can be moved from tools/ to tools/lint/deep/nested/ and still correctly identify the root without code changes.

3.2 The Skill Registry Violation

The file SKILL.md currently resides at the repository root.

- **Audit Finding:** SKILL.md contains the "Skills Promotion Template," a "Harvest Engine workflow," and logs for candidates and promotions.
- **Violation:** The TRUTH.md Filing Doctrine states:

    - ops/ = Run
    - docs/ = Explain
    - Root = Constitution & State (TRUTH, AGENTS, SoP, TASK)
- **Analysis:** SKILL.md is neither Constitution nor State. It is procedural documentation and a registry of capabilities. While SoP.md tracks the history of the *system rules*, SKILL.md tracks the history of *learned capabilities*. Placing it at the root dilutes the authority of the Canon. It belongs in the library.
- **Remediation:** SKILL.md must be moved to docs/library/SKILLS.md. This aligns with the filing doctrine by grouping "Explanation" (Skills) with the library, while keeping the root clean for "State" and "Law."

3.3 Documentation Debt and Redundancy

The audit identifies several files that violate the "Economy of Truth" by creating redundant or conflicting sources of information.

1. **ops/lib/project/user-STELA.md**: This file contains user-level configurations (e.g., IDE settings).
    - **Audit Finding:** "User-level" configurations create hidden state. A project should be fully defined by its own STELA.md and the system canon. Personal preferences pollute the reproducibility of the "Factory." This file represents "Drift" and must be deleted.
2. **ops/lib/project/INTEGRATION.md**: This file describes the integration contract.
    - **Audit Finding:** Since ops/bin/project will implement the integration logic via the work command composition, the text file is redundant. The code itself becomes the documentation of the contract. This file represents "Documentation Debt" and must be deleted.
3. **docs/library/INTEGRATION.md**: This is the operator guide for integration.
    - **Audit Finding:** It essentially points to the file above. If the logic is implemented in the binary, the manual entry in docs/library/MANUAL.md suffices. "How to run integration" becomes simply ops/bin/project work --task=.... This file must be deleted.

4. Drift Detection: The Agent Injection Fallacy

The user has explicitly flagged the PDF's proposal regarding agent injection as "Drift." This section analyzes why that proposal was flawed and articulates the corrected "Ontological Map" that DP-OPS-0022 must implement.

4.1 Rejection of the Replacement Strategy

The PDF proposed: *"The extracted role replaces the generic 'Contractor' definition in the system prompt."*

The Flaw: Replacing the "Contractor" definition removes the baseline constraints. The

"Contractor" definition in AGENTS.md (via the system prompt) carries the weight of the "Non-Negotiable Behavioral Constraints" (No contractions, Literalism, Anti-Drift). If we replace this with R-AGENT-06 (Security Reviewer), which focuses solely on security vulnerabilities, the agent effectively "forgets" the Constitutional laws. It might start using contractions or making subjective reports because its new "system prompt" doesn't forbid it.

The Correction: The relationship is **Additive, not Substitutive**. The specific agent role is injected *alongside* the Constitution, not *instead of* it. The prompt construction must be:

1. **Constitution (AGENTS.md):** You are a Contractor. You follow these rules (No contractions, etc.).
2. **Citizen (R-AGENT-06):** Your job is Security Reviewer. You use these tools.

4.2 The Ontological Map: Constitution vs. Citizen

The user's directive establishes a clear political metaphor for the system architecture:

- **AGENTS.md (Constitution):** This defines the immutable laws of the environment. It applies to *every* entity, human or AI. It defines the "Rights" (what access levels exist) and the "Responsibilities" (Behavioral Logic Standards). It is the supreme law of the repository.
- **docs/library/agents/ (Citizen):** These files define the specific actors who operate within the environment. R-AGENT-01 (Architect) and R-AGENT-06 (Security Reviewer) are "Citizens" of the Stela system. As citizens, they have different jobs, but they are all equally subject to the Constitution. A Security Reviewer cannot violate the "No Contractions" rule any more than an Architect can.

Enforcement Logic: The ops/bin/open script must be refactored to enforce this hierarchy. It must load AGENTS.md *first*, establishing the constraints. It then loads the requested agent file *second*, injecting the specific expertise. The prompt must explicitly state that the Agent file *cannot* override AGENTS.md.

4.3 The Operational Map: Object vs. Class

The relationship between TASK.md and docs/library/tasks/ was previously ambiguous. The new directive clarifies this using an Object-Oriented Programming (OOP) metaphor:

- **docs/library/tasks/ (Class):** These are the "Immutable Recipes" or "Standard Operating Procedures (SOPs)." For example, B-TASK-01 (Code Review) defines the *abstract* process of reviewing code. It is a template (Class) that exists in the library.
- **TASK.md (Object/Instance):** This is the "Live Workspace." When an operator wants to perform a code review, they copy the recipe from B-TASK-01 into TASK.md. At that moment, TASK.md becomes an *instance* (Object) of the B-TASK class. It is "ephemeral-to-the-goal"—it changes state, logs progress, and is eventually archived or overwritten by the next task.

Usage: The operator does not "edit" the library file to do work. They "instantiate" the library file into the active workspace. This preserves the sanctity of the library (the "Class" definition) while allowing the active task (the "Object" state) to evolve.

5. Architectural Remediation Strategy

To address the forensic findings and enforce the corrected Ontological Map, the following architectural changes will be codified in DP-OPS-0022.

5.1 Git-Anchored Script Robustness (The Fix)

To resolve the "Red Gate" and ensure "Location Independence," all operational scripts in tools/ must implement Dynamic Root Resolution. We cannot rely on pwd or relative paths.

The Logic:

The script must ask Git: "Where is the root of this tree?" If Git answers, that answer is the Truth. If Git cannot answer (e.g., in a stripped container), the script must fall back to a heuristic search for a canonical anchor file (TRUTH.md).

Proposed Code Pattern:

Bash

```
# Robust Repo Root Resolution
if command -v git >/dev/null 2>&1 && git rev-parse --is-inside-work-tree >/dev/null 2>&1; then
    REPO_ROOT="$(git rev-parse --show-toplevel)"
else
    # Fallback for non-git environments (e.g., rigid container mounts)
    # Assumes script is at tools/lint/script.sh (2 levels down)
    SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE}")" && pwd)"
    # Attempt to resolve root by looking for TRUTH.md
    if]; then
        REPO_ROOT="$(cd "${SCRIPT_DIR}/../../" && pwd)"
    elif]; then
        REPO_ROOT="$(cd "${SCRIPT_DIR}/.." && pwd)"
    else
        echo "ERROR: Could not locate REPO_ROOT (TRUTH.md not found)." >&2
        exit 1
    fi
fi
```

This pattern must be applied to tools/lint/context.sh, tools/lint/truth.sh, and tools/lint/library.sh. Furthermore, tools/lint/truth.sh currently uses git ls-files which is sensitive to the current working directory. It must be updated to explicitly cd "$REPO_ROOT" before execution to ensure the file list is generated relative to the root.

5.2 Consolidation of the Project Factory

The fragmentation of the "Library Trinity" (Agents, Tasks, Skills) must be resolved.

1. **Move SKILL.md**: Relocate from root to docs/library/SKILLS.md.
2. **Update ops/lib/scripts/skill.sh**: Point the SKILL_FILE variable to the new location.
3. **Update llms.txt**: Ensure the entry point references the new location.
4. **Update docs/library/INDEX.md**: Reflect the move and ensure the "Registry" is accurate.

This consolidation physically enforces the ontological distinction: "State" (SoP) is at the root; "Knowledge" (Skills) is in the library.

5.3 Epistemic Hygiene

To reduce "Context Pollution," we must purge the redundant files identified in Section 3.3 (user-STELA.md, INTEGRATION.md x2). Deleting these files reduces the "Token Cost" of the repository and eliminates conflicting sources of truth. The logic of integration belongs in the *code* (ops/bin/project), not in static markdown files that are prone to rot.

6. The Dispatch Packet (DP-OPS-0022)

The following Dispatch Packet is the executable artifact of this report. It is drafted in strict adherence to the TASK.md headers and the Stela Standard (A-E), incorporating the specific logic overrides mandated by the user.

DP-OPS-0022: Fix Repo-Gates & Consolidate Project System

3.1 Freshness Gate (Must Pass Before Work)

Base Branch: main

Required Work Branch: work/fix-gates-consolidate-2026-02-04

Base HEAD: f2258f404

Required local re-check (worker runs; paste results in RESULTS):

- git rev-parse --abbrev-ref HEAD
- git rev-parse HEAD
- git status --porcelain

Worker must already be on the **Required Work Branch** listed above (no branch creation or switching).

STOP if any mismatch.

STOP if the Required Work Branch is missing.

STOP if told to create or switch branches.

3.2 Required Context Load (Read Before Doing Anything)

Worker must confirm loaded before acting:

- TRUTH, AGENTS, SoP, CONTEXT, MAP
- Plus any DP-scoped files listed here:
  - tools/lint/context.sh
  - tools/lint/truth.sh
  - tools/lint/library.sh
  - ops/lib/scripts/skill.sh
  - docs/library/INDEX.md

(OPEN is for Integrator/DP Writer state refresh; Worker does not read OPEN).

DISPOSABLE ARTIFACTS (chat logs, etc.) MUST NOT BE REFERENCED OR INCLUDED.

3.3 Scope and Safety

- **Objective:** Fix linter path resolution to green repo-gates; move SKILL.md to library; remove redundant documentation.
- **Non-Goals (optional, drift-killer):**
  - No changes to project payloads in projects/.
  - No changes to AGENTS.md text (only path references in other files).

Target Files allowlist (hard gate)

- tools/lint/context.sh
- tools/lint/truth.sh
- tools/lint/library.sh
- tools/verify.sh
- SKILL.md (move to docs/library/SKILLS.md)
- (new) docs/library/SKILLS.md
- ops/lib/scripts/skill.sh
- llms.txt
- ops/lib/project/user-STELA.md (delete)
- ops/lib/project/INTEGRATION.md (delete)
- docs/library/INTEGRATION.md (delete)
- docs/library/INDEX.md
- SoP.md

Allowlist rule: exact paths only; use (new) prefix only when the DP explicitly allows new files.

- **Conditional Requirements:** If canon/governance surfaces (e.g., TASK.md) are modified, SoP.md MUST be included in the allowlist and updated.
- **Stop Condition (scope):** If any required change falls outside the allowlist, **STOP** and report.
- **Forbidden (unless DP explicitly overrides):**
  - No edits to .github/
  - No edits to projects/ unless explicitly authorized
  - No renames/moves/deletes unless explicitly authorized
- **Worker Git Authority (non-negotiable):**
  - Workers do not create/switch branches.
  - Workers do not stage/commit/merge/push.
  - Workers only return diffs/results/receipt.
- **Precedence:** If any conflict exists, **TRUTH.md wins**.
  If unclear, STOP and ask.
- **Strict Stop Conditions (generic):**
  - STOP if any placeholder text remains in final outputs ([...], TODO, TBD, ...).
  - STOP if required DP inputs are missing (do not guess).
  - STOP if IDs / naming constraints in the DP are inconsistent (when applicable).

3.4 Execution Plan (A-E Canon)

3.4.1 State

- **Context:** The repository is currently failing repo-gates because linter scripts in tools/lint/ use relative path logic (cd../..) that is fragile and incorrect given their depth. Additionally,

the "Library Trinity" (Agents, Tasks, Skills) is fragmented, with SKILL.md at the root and redundant integration docs causing epistemic drift.

- **Drift:** tools/lint/context.sh looks for manifest at tools/ops/... instead of root. SKILL.md violates Filing Doctrine by being at root. INTEGRATION.md files are redundant to the binary.
- **Desired State:** All linters use git rev-parse --show-toplevel for robust root resolution. SKILL.md is moved to docs/library/SKILLS.md. Redundant files are deleted.

3.4.2 Request

Numbered tasks (concrete, no vibes):

1. **Refactor Linters:** Update tools/lint/context.sh, tools/lint/truth.sh, and tools/lint/library.sh to use the robust git rev-parse --show-toplevel pattern for REPO_ROOT resolution. Ensure fallback logic exists for non-git environments.
2. **Fix Truth Linter:** In tools/lint/truth.sh, ensure the script explicitly cd "$REPO_ROOT" before running git ls-files to guarantee correct file discovery regardless of invocation directory.
3. **Relocate SKILL.md:** Move SKILL.md to docs/library/SKILLS.md.
4. **Update Skill Script:** Modify ops/lib/scripts/skill.sh to point SKILL_FILE to the new location docs/library/SKILLS.md.
5. **Update LLMS Entry Point:** Update llms.txt to point to docs/library/SKILLS.md instead of SKILL.md (if applicable) or ensure the small bundle references the new path.
6. **Purge Documentation Debt:** Delete ops/lib/project/user-STELA.md, ops/lib/project/INTEGRATION.md, and docs/library/INTEGRATION.md.
7. **Update Registry:** Update docs/library/INDEX.md to remove the INTEGRATION.md entry and update the skills-template entry to point to docs/library/SKILLS.md.
8. **Update SoP:** Log the governance changes (Linter fix + Library consolidation).

Required Content (optional; use when doc/governance changes demand exact text/format):

- Ensure the Linter fix uses the following pattern:

```Bash
if command -v git >/dev/null 2>&1 && git rev-parse --is-inside-work-tree >/dev/null 2>&1;
then
  REPO_ROOT="$(git rev-parse --show-toplevel)"
else
  # Fallback logic
  SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE}")" && pwd)"
  #... (logic to find TRUTH.md)...
fi
```

3.4.3 Changelog

- **Log (human-readable, 1–6 bullets):**
  - Refactored tools/lint/*.sh to use git rev-parse for robust root resolution (Fixes Red Gate).
  - Moved SKILL.md to docs/library/SKILLS.md to align with Filing Doctrine.
  - Deleted redundant INTEGRATION.md and user-STELA.md files.
  - Updated ops/lib/scripts/skill.sh and docs/library/INDEX.md to reflect paths.

3.4.4 Patch / Diff

- **Format:** Unified diff (preferred) or anchored snippets.

3.4.5 Receipt (Required)

Verification (MUST RUN; or report NOT RUN + reason + risk)

- ./ops/bin/dump --scope=platform
- If dump refiners are used, include this fallback line in the DP:
  - Fallback:./ops/bin/dump --scope=platform --format=chatgpt --out=auto --bundle
- **Zero-Byte Check:** Verify dump is not empty (test -s <path>).
- Context lint (use repo-canonical command):
  - Prefer: bash tools/lint/context.sh
  - If not found and required: **STOP** and ask (do not invent a substitute)
- Truth lint (when TRUTH/canon surfaces touched; use repo-canonical command):
  - Prefer: bash tools/lint/truth.sh
  - If not found and required: **STOP** and ask (do not invent a substitute)

Skill capture (required during normal DP processing)

- Workflow: run ops/lib/scripts/skill.sh harvest to auto-generate a draft with provenance and semantic collision checks.
- Review: inspect the draft in storage/handoff/ for heuristic accuracy (Hot Zone and High Churn). Refine Invocation guidance and Solution.
  Do not edit the Provenance block.
- Promote: run ops/lib/scripts/skill.sh promote <draft_path> or log "no new skill promoted" with a rationale in RESULTS.
- Legacy capture is forbidden. Do not manually write skill files; the provenance block is required.
- If a DP requires skill capture, the DP Target Files allowlist must include SKILL.md (or docs/library/SKILLS.md).
- Proof required in RESULTS:
  - Harvest-only: command invoked plus draft path and ls storage/handoff/ output.
  - Promotion: command invoked plus grep -n "Promotion Packet:" docs/library/SKILLS.md and grep -n "docs/library/skills/S-LEARN-"

docs/library/INDEX.md.

Artifact bundle (required directory)

- storage/handoff/

Proof bundle checklist (required; naming pattern over exact names)

- storage/handoff/DP-OPS-0022-RESULTS.md (required)
- storage/handoff/OPEN-PORCELAIN-[work-branch]-[hash].txt (required)
- storage/handoff/OPEN-[work-branch]-[hash].txt (required)
- storage/dumps/dump-full-... (or platform scope as applicable)
- git diff --name-only (include in RESULTS)
- git diff --stat (include in RESULTS)
- NEXT: one single action (include in RESULTS)

MANDATE: No receipt/proof bundle = **AUTOMATIC DISAPPROVAL**.

Closeout routine receipts not being produced CONSISTENTLY INCLUDING PORCELAIN, DUMP AND OPEN is a failure condition.

Partial receipts (e.g., missing OPEN artifacts when a session was started) are considered failures.

REQUIRED OUTPUT

MUST include a "Status" block at the top of RESULTS:

- Scope summary: (1 sentence validating the loop)
- Tracked change: (What was edited)
- Verification: (Output of proofs)

RESULTS file must include (minimum)

- Verification results (RUN/NOT RUN + reason + risk)
- Patch/diff proof bundle (status/diff list or direct diff)
- Receipt pointers (OPEN + PORCELAIN + DUMP outputs/paths)
- git diff --name-only output
- git diff --stat output
- NEXT: one single action

Storage artifact flow (local-only, untracked):

- OPEN and OPEN-PORCELAIN outputs land in storage/handoff/.
- Dump bundles land in storage/dumps/.
- Results receipts land in storage/handoff/DP-OPS-0000-RESULTS.md (pattern).
- Operator stores DP drafts under storage/dp/intake/ and moves or copies to storage/dp/processed/ after completion (or when reusing a previously executed DP).

- These are retention paths only; they are not worker prerequisites unless the DP explicitly says so.

---

4. Closeout (Mandatory)

Worker closeout duties (end of session)

- [ ] Produce/Update: storage/handoff/DP-OPS-0022-RESULTS.md
- [ ] If canon/governance surfaces were changed, include **SoP.md** in scope and update it in the same PR slice.
- [ ] Run ops/bin/prune (hygiene).
- [ ] Run ops/bin/llms (refresh context bundles).
- [ ] Ensure proof bundle checklist is satisfied (above).
  (No bundle = DISAPPROVE.)
- [ ] Fill out the Mandatory Closing Block (see below).
- [ ] Append a **TASK Work Log** entry (below) capturing:
  - what changed (high level)
  - gates/verification outcome (pass/fail or not run + reason)
  - blockers (if any)
  - **NEXT:** one single action

Mandatory Closing Block (Varied Wording Required)

Varied Wording provision: Each entry must use meaningfully distinct wording;

copy or minor tense changes are not acceptable. Entry 4 must differ from Entry 1.

1. Primary Commit Header (plaintext)
2. Pull Request Title (plaintext)
3. Pull Request Description (markdown)
4. Final Squash Stub (plaintext) (Must differ from #1)
5. Extended Technical Manifest (plaintext)
6. Review Conversation Starter (markdown)

Integrator/operator closeout duties

- [ ] Review diff vs DP scope/forbidden zones
- [ ] Run repo-gates / required checks
- [ ] Commit/push/open PR/merge (operator-controlled)

---

7. Further Findings and Next Phase Integration

The execution of DP-OPS-0022 will establish a "Green Gate" baseline, a prerequisite for

advanced agent orchestration. The "Constitution vs. Citizen" model clarifies the role of AI: The AI is a citizen (Worker) bound by the Constitution (Rules), utilizing Skills (Tools) to execute Tasks (Recipes). This taxonomy eliminates the ambiguity of "who is in charge" (The Operator/Constitution is always in charge).

The "Red Gate" failure highlights that tools/ scripts must be treated as production code with the same rigor as the projects/ payload. The shift to git-anchored paths ensures that the "Factory" itself is not brittle.

Next Phase (Phase 3):

Upon successful merge of DP-OPS-0022, the system will be ready for **DP-OPS-0023: Agent Specialization**. This phase will implement the ops/bin/project work --agent=X --task=Y logic, leveraging the now-consolidated "Library Trinity." The stable docs/library/SKILLS.md and docs/library/agents/ paths will serve as the reliable data source for this dynamic context injection. The "Object-Class" relationship of TASK.md will be codified in the project binary, allowing operators to "instantiate" tasks directly from the library.